

Data Types

Various Data Types in Programming

The numbers handled by modern computers are of the following types:

- **Integers** – numbers devoid of the fractional part
- **Floating Points** – or floats are numbers that contain or can contain the fractional part

These numbers differ significantly in how they are stored in computer memory and the range of acceptable values. The characteristic of the numeric value that determines its kind, range, and application is called the **type**.

Integers

An **integer** is a string of digits that make up the number. It must not be interjected with any characters that are not digits. For example, the number *eleven million one hundred and eleven thousand one hundred and eleven* can be written as 11,111,111 or 11.111.111, or even like 11 111 11.

This provision gives it readability, especially when the number consists of many digits. However, Python does not accept this as it is **prohibited**. What Python does allow, though, is the use of **underscores** in numeric literals.

Therefore, the number can either be 11111111 or 11_111_111.

Negative numbers in Python are written by adding a minus (-), such as -11111111 or -11_111_111. Positive numbers do not need to be preceded by the plus sign, but it is permissible.

Octal and *hexadecimal* numbers are two (2) additional conventions in Python that are unknown to the world of mathematics.

It will be treated as an **octal** value if an integer number is preceded by an 00 or 0o prefix (zero-o). This means that the number must contain digits taken from the [0..7] range only.

For example, 0o123 is an **octal** number with a (decimal) value equal to 83. The print() function does the conversion automatically.

On the other hand, a number will be treated as a **hexadecimal value** if it is preceded by the prefix 0x or 0X (zero-x)

For example, 0x123 is a **hexadecimal** number with a (decimal) value equal to 291. The print() function can also manage these values.

Floats

A **float** is designed to represent and store numbers with a **non-empty decimal fraction**. These numbers have or may have, a fractional part after the decimal point. For example, *two and a half* or *minus zero point four* is read as 2.5 and -0.4 by a computer.

It is also important to note that numbers must not contain any commas. Python does not accept the comma as it has its reserved meaning in Python.

Removing the zero if it is the only digit in front of or after the decimal point is possible. Thus, 0.4 can be written as .4, and 4.0 could be written as 4. and have its type or value unchanged.

Essentially, the point (.) makes a number a float. But also, the letter E/e, which represents **exponents** incorporated into a number string such as 3E8. The *exponent* (value after E) has to be an integer, while the *base* (value in front of the E) may be an integer.

Exponents are coded this way:

The value of a physical constant called Planck's constant is 6.62607×10^{-34} . To code this, it will be formatted as 6.62607E-34.

It also must be noted that the chosen form of coding float values does not mean that Python will present it in the same way. This language may sometimes choose a different notation.

For example, the following float literal: `0.0000000000000000000001` will result in `1e-22` when run using the `print()` function in Python, as it always chooses the more economical form of the number's presentation.

Strings

These are used when processing texts and not numbers. A string is incomplete without quotes ("").

For example: "Today, I start changing for good!"

On the flip side, assume that words inside a string need to be quoted, such as *She screamed, "You need to calm down" at the top of her lungs, which is ironic.*

There are two (2) possible solutions. The first is based on the concept of the **escape character**, which is played by the backslash (\). The backslash can escape quotes, too. A quote preceded by a backslash changes its meaning – it is not a delimiter but just a quote.

The string becomes:

```
print("She screamed, \"You need to calm down\" at the top of her lungs, which is  
ironic.")
```

The second solution is to use an apostrophe (') instead of a quote. Either of these characters may delimit strings, but it should be consistent in such a way that:

- If a string opens with a quote, it must also be closed with a quote
- If a string starts with an apostrophe, it must also end with an apostrophe

The string becomes:

```
print('She screamed, "You need to calm down" at the top of her lungs, which is  
ironic.')
```

To embed an apostrophe into a string, there are also two (2) possible ways:

Example string: *I'm not going to give up.*

It can be written as:

```
print("I'm not going to give up.")
```

or

```
print('I\'m not going to give up.')
```

Boolean Values

Each time Python is asked if one number is greater than another, it always results in the creation of some specific data, which is a **Boolean value**.

The name comes from George Boole (1815-1864), the author of the fundamental work, *The Laws of Thought*, which contains the definition of **Boolean algebra** - a part of algebra that makes use of only two distinct values: True and False, denoted as 1 and 0.

Python is considered a binary reptile as True and False have strict denotations. The results cannot be changed, and the symbols must be taken as they are, including case-sensitivity.

Reference:

Edube (n.d.). *Python essentials 1*. Retrieved from <https://edube.org/learn/pe-1/python-literals-30/>